

Geometric transformations overview

Below is a **clean, structured, and detailed explanation** of the entire passage, written as if you are studying **Chapter 7: Geometric Transformations** for exams, interviews, or conceptual clarity in computer graphics. Nothing essential is skipped.

1. What are Geometric Transformations?

In computer graphics, once a scene is described using **graphics primitives** (points, lines, polygons, curves) and rendered using raster algorithms (like scan-line algorithms), the next major requirement is to **change how objects appear in a scene**.

This is done using **geometric transformations**.

Definition

Geometric transformations are mathematical operations applied to the geometric description of an object to change its:

- Position
- Orientation
- Size

without changing the object's intrinsic shape (unless scaling is non-uniform).

These transformations are fundamental in:

- Viewing pipelines (world coordinates → device coordinates)
 - CAD systems
 - Computer animation
 - Simulation
 - Interactive graphics
-

2. Geometric vs Modeling Transformations

Some graphics systems distinguish between **modeling transformations** and **geometric transformations**, while others treat them as the same.

Modeling Transformations

Used to **construct complex objects hierarchically**.

Example:

- An aircraft is composed of wings, fuselage, tail, engines

- Each component has its **own local coordinate system**
- Modeling transformations specify how each component fits into the whole

This forms a **hierarchical structure**, where:

- Lower-level components are positioned relative to higher-level components
- Transformations accumulate through the hierarchy

Geometric Transformations

Used to:

- Move objects around a scene
- Animate objects
- Change viewing angle
- Simulate camera motion

In practice:

- Many graphics APIs (OpenGL, Java graphics, etc.) use **one unified transformation system** for both purposes

3. Basic Two-Dimensional Geometric Transformations

In 2D graphics, the fundamental transformations are:

1. Translation
2. Rotation
3. Scaling

Additional (sometimes included):

- Reflection
- Shearing

Understanding 2D transformations is essential because:

- 3D transformations are direct extensions
- Viewing transformations use the same mathematics

4. Two-Dimensional Translation

Concept

Translation moves an object **without changing its size or orientation**.

Every point of the object is shifted by the **same displacement vector**.

This makes translation a **rigid-body transformation**.

Mathematical Formulation

Given an original point:

$$(x, y)$$

Translation vector:

$$(t_x, t_y)$$

New point:

$$(x_r, y_r)$$

The translation equations are:

$$x_r = x + t_x$$

$$y_r = y + t_y$$

The pair (t_x, t_y) is called the **translation vector** or **shift vector**.

Vector-Matrix Representation

Using column vectors:

$$P = \begin{bmatrix} x \\ y \end{bmatrix}, \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}, \quad P_r = \begin{bmatrix} x_r \\ y_r \end{bmatrix}$$

The translation equation becomes:

$$P_r = P + T$$

Translation of Objects

- **Line:** translate both endpoints
- **Polygon:** translate every vertex
- **Circle / ellipse:** translate center
- **Spline / curve:** translate control points

All points move along **parallel straight-line paths**.

5. Two-Dimensional Rotation

Concept

Rotation moves points along **circular paths** around a fixed point.

In 2D:

- Rotation axis is perpendicular to the xy-plane (along the z-axis)
- Rotation is defined by:
 - Angle θ
 - Pivot point (rotation point)

Positive angle $\rightarrow$ counterclockwise rotation

Negative angle $\rightarrow$ clockwise rotation

6. Rotation About the Origin

Polar Coordinate Interpretation

For a point at distance r from the origin:

$$x = r \cos \phi$$

$$y = r \sin \phi$$

After rotation by angle θ :

$$x_r = r \cos(\phi + \theta)$$

$$y_r = r \sin(\phi + \theta)$$

Using trigonometric identities:

$$x_r = x \cos \theta - y \sin \theta$$

$$y_r = x \sin \theta + y \cos \theta$$

Matrix Form

$$P_r = R \cdot P$$

where

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

This is the **2D rotation matrix**.

Important Convention

Modern graphics systems (OpenGL, Java, PHIGS, GKS) use:

- **Column vectors**
- **Pre-multiplication**

Older systems sometimes used row vectors, which changes multiplication order.

7. Rotation About an Arbitrary Pivot Point

Most rotations are **not about the origin**.

Let pivot point be:

$$(x_p, y_p)$$

Rotation steps conceptually:

1. Translate pivot to origin
2. Rotate
3. Translate back

The resulting equations are:

$$x_r = x_p + (x - x_p) \cos \theta - (y - y_p) \sin \theta$$

$$y_r = y_p + (x - x_p) \sin \theta + (y - y_p) \cos \theta$$

These equations contain:

- Multiplicative (rotation) terms
 - Additive (translation) terms
-

Rotation of Objects

- **Line:** rotate both endpoints
- **Polygon:** rotate all vertices
- **Circle:**
 - If rotating about center → no visible change
 - If rotating about another point → center moves in a circular arc
- **Ellipse:**
 - Rotate axes or control parameters

Rotation is a **rigid-body transformation**.

8. Two-Dimensional Scaling

Concept

Scaling changes the **size** of an object.

Unlike translation and rotation, scaling can:

- Change object proportions
- Change distance from origin

Scaling About the Origin

Given scaling factors:

$$s_x, s_y$$

Equations:

$$x_r = x \cdot s_x$$

$$y_r = y \cdot s_y$$

Matrix form:

$$P_r = S \cdot P$$

where:

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Interpretation of Scaling Factors

- $s_x = s_y = 1$: no change
- $0 < s < 1$: shrink
- $s > 1$: enlarge
- $s_x = s_y$: uniform scaling
- $s_x \neq s_y$: differential (non-uniform) scaling
- Negative values:
 - Scale + reflect across axes

Scaling relative to the origin also **moves objects** closer to or farther from the origin.

9. Scaling About a Fixed Point

To prevent unwanted movement, scaling is often done relative to a **fixed point**:

$$(x_f, y_f)$$

The distance from each point to the fixed point is scaled.

Equations:

$$x_r - x_f = (x - x_f)s_x$$

$$y_r - y_f = (y - y_f)s_y$$

Rewritten:

$$x_r = xs_x + x_f(1 - s_x)$$

$$y_r = ys_y + y_f(1 - s_y)$$

Here:

- Multiplicative terms → scaling
- Additive terms → repositioning

This is conceptually similar to rotation about a pivot point.

Scaling of Objects

- **Polygon:** scale each vertex
- **Circle:** scale radius
- **Ellipse:** scale major/minor axes
- **Curves:** scale control points

10. Why Matrix Formulation Matters

Although translation, rotation, and scaling were initially shown using separate equations, modern graphics systems use:

- **Homogeneous coordinates**
- **Matrix multiplication**
- **Transformation composition**

Advantages:

- Combine multiple transformations
- Efficient hardware implementation
- Clean pipeline integration

This is discussed in later sections of the chapter.

Summary Table

Transformation	Preserves Shape	Preserves Size	Preserves Orientation
Translation	Yes	Yes	Yes
Rotation	Yes	Yes	Yes
Scaling	Yes	No	Yes (if uniform)

If you want, I can next:

- Convert this into **exam-ready notes**
- Explain **homogeneous coordinates**
- Show **OpenGL transformation pipeline**
- Give **worked numerical examples**
- Provide **Python/OpenGL code examples**

Just tell me what you want next.

Below is a **thorough, concept-by-concept explanation** of **Section 2: Matrix Representations and Homogeneous Coordinates**, written for deep understanding and exam readiness. I will strictly stay explanatory, detailed, and technical.

1. Why Matrix Representations Are Needed

In real graphics applications, **single transformations are rarely applied in isolation**.

Typical scenarios:

- **Animation:** An object is translated, then rotated, at every frame.
- **CAD / design:** Objects are scaled, rotated, and positioned relative to each other.
- **Viewing pipeline:** World coordinates → camera coordinates → screen coordinates involves many transformations.

If transformations are applied **one after another**, using equations like:

$$P' = M_1 \cdot P + M_2$$

then:

- Each step produces intermediate coordinates
- These intermediate results must be stored and recomputed
- Performance becomes inefficient

The goal is:

Combine multiple transformations into a single transformation

so that final coordinates are computed directly from the original ones.

To achieve this efficiently, **all transformations must be expressed purely as matrix multiplications**, with **no matrix addition**.

This is where **homogeneous coordinates** become essential.

2. Limitation of the Original Matrix Form

From earlier sections, all 2D transformations were written as:

$$P' = M_1 \cdot P + M_2 \quad (\text{Equation 15})$$

Where:

- P, P' are 2×1 column vectors
- M_1 is a 2×2 matrix (rotation/scaling)
- M_2 is a 2×1 vector (translation terms)

Problem:

Matrix **addition** prevents easy composition.

Example:

If you want to apply:

1. Scaling
2. Rotation
3. Translation

You must compute:

- Intermediate coordinates after scaling
- Then rotate those results
- Then translate again

This is **inefficient** and **hard to pipeline in hardware**.

3. Core Idea of Homogeneous Coordinates

The key idea is:

Convert translation into matrix multiplication by increasing the dimension of the matrices

This is done by:

- Expanding 2D coordinates from **2 components** → **3 components**
- Expanding transformation matrices from **2×2** → **3×3**

This allows:

- All transformations to be written as:

$$P' = M \cdot P$$

- Easy composition using matrix multiplication
- Efficient GPU implementation

4. Homogeneous Coordinate Representation

A 2D Cartesian point:

$$(x, y)$$

is represented in homogeneous form as:

$$(x_h, y_h, h)$$

where:

$$x = \frac{x_h}{h}, \quad y = \frac{y_h}{h} \quad (\text{Equation 16})$$

Important Properties:

- $h \neq 0$
- Many homogeneous representations map to the same Cartesian point
- Example:
 - $(2, 4, 2) \rightarrow (1, 2)$
 - $(10, 20, 10) \rightarrow (1, 2)$

Thus, homogeneous coordinates are **not unique**.

5. Why Choose $h = 1$?

For geometric transformations:

- Any nonzero value of h is valid
- Choosing $h = 1$ simplifies calculations

So the standard representation becomes:

$$(x, y) \rightarrow (x, y, 1)$$

This is why:

- All 2D graphics APIs internally use (x, y, 1)
- It allows translation terms to appear directly in matrices

Other values of h are needed mainly in **3D perspective projection**, not here.

6. Mathematical Meaning of “Homogeneous”

The term *homogeneous* comes from algebra.

If a Cartesian equation:

$$f(x, y) = 0$$

is converted to homogeneous form:

$$f(x_h, y_h, h) = 0$$

then multiplying all variables by a constant v :

$$(vx_h, vy_h, vh)$$

does **not change the equation**, because the factor v cancels out.

This property is what allows:

- Infinite representations of the same point
 - Clean transformation mathematics
-

7. General Transformation Framework

With homogeneous coordinates:

- Points $\rightarrow 3 \times 1$ column vectors
- Transformations $\rightarrow 3 \times 3$ matrices

All transformations follow:

$$P' = M \cdot P$$

No addition terms are required.

This is the **standard representation used in graphics systems**.

8. Two-Dimensional Translation Matrix

Original translation equations:

$$x' = x + t_x$$

$$y' = y + t_y$$

Using homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (\text{Equation 17})$$

Key Insight:

- Translation values appear in the **third column**
- Multiplication by 1 (homogeneous coordinate) injects translation

This matrix is written compactly as:

$$P' = T(t_x, t_y) \cdot P$$

If parameters are obvious, we simply write:

$$T$$

9. Two-Dimensional Rotation Matrix (About Origin)

Original rotation equations:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

Homogeneous matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (\text{Equation 19})$$

Written compactly as:

$$P' = R(\theta) \cdot P$$

Notes:

- Rotation is about the **origin**
- No translation terms → third column is zeros
- Rotation matrix occupies the top-left 2×2 submatrix

10. Rotation About Arbitrary Pivot (Conceptual Note)

Some graphics libraries:

- Only provide rotation about the origin
- Require users to do:
 1. Translate pivot to origin
 2. Rotate
 3. Translate back

Other libraries:

- Accept pivot coordinates
- Internally generate the full 3x3 matrix

Homogeneous matrices make both approaches equivalent.

11. Two-Dimensional Scaling Matrix

Scaling equations:

$$x' = s_x x$$

$$y' = s_y y$$

Homogeneous matrix:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (\text{Equation 21})$$

Compact form:

$$P' = S(s_x, s_y) \cdot P$$

12. Scaling About Arbitrary Fixed Point

If scaling must occur relative to a fixed point:

- Some systems require:
 1. Translate fixed point to origin
 2. Scale
 3. Translate back
- Others generate a direct homogeneous matrix

Again, homogeneous coordinates allow **both methods**.

13. Major Advantage of Homogeneous Matrices

Transformation Composition

If you apply:

1. Scaling $\rightarrow S$
2. Rotation $\rightarrow R$
3. Translation $\rightarrow T$

Final transformation:

$$P' = T \cdot R \cdot S \cdot P$$

This single matrix:

- Can be computed once
- Applied to all points
- Eliminates intermediate results
- Is ideal for hardware acceleration

14. Why This Is Critical in Graphics Systems

Homogeneous coordinates:

- Enable transformation pipelines
- Enable hierarchical modeling
- Enable camera and projection transformations
- Are the foundation of OpenGL, DirectX, Vulkan

Without homogeneous coordinates:

- Translation cannot be merged with other transformations
- Efficient rendering pipelines are impossible

Final Conceptual Summary

Aspect	Without Homogeneous	With Homogeneous
Translation	Requires addition	Matrix multiply
Composition	Difficult	Simple
Efficiency	Low	High
Hardware support	Poor	Excellent
Graphics pipelines	Not feasible	Standard

If you want next:

- **Numerical worked examples**
- **Matrix composition order explanation**
- **Why order of multiplication matters**
- **Connection to 3D transformations**
- **OpenGL transformation stack**

tell me exactly what to continue with.

Below is a **complete, structured, and in-depth explanation** of **Section 4: Two-Dimensional Composite Transformations**, written to build strong conceptual understanding for **exams, interviews, and practical graphics programming**. Every idea is unpacked carefully, with emphasis on *why* things work, not just formulas.

1. What Are Composite Transformations?

In real graphics systems, objects are **rarely transformed using a single operation**.

Typical examples:

- Rotate an object about a specific pivot point
- Scale an object about its center, then translate it
- Animate an object by repeatedly rotating and translating it
- Perform viewing transformations (camera movement)

Applying each transformation separately to every point is inefficient.

Core Idea

Instead of applying transformations one by one to points, we:

1. **Combine (concatenate) all transformation matrices**
2. Form **one composite transformation matrix**
3. Apply it once to all points

This approach is called **composite (or concatenated) transformations**.

2. Why Matrix Concatenation Works

With homogeneous coordinates:

- Points are column vectors

- Transformations are 3×3 matrices
- All transformations are applied via matrix multiplication

If we apply two transformations M_1 followed by M_2 to a point P :

$$P' = M_2 \cdot (M_1 \cdot P)$$

Using associativity of matrix multiplication:

$$P' = (M_2 \cdot M_1) \cdot P$$

So instead of transforming points twice:

- We compute the **composite matrix**

$$M = M_2 \cdot M_1$$

- Then apply it once:

$$P' = M \cdot P$$

This is **faster, cleaner, and hardware-friendly**.

3. Composite Two-Dimensional Translations

Translation Recap

A translation by (t_x, t_y) is represented by:

$$T(t_x, t_y)$$

Two Successive Translations

Suppose we apply:

1. Translation $T(t_{1x}, t_{1y})$
2. Translation $T(t_{2x}, t_{2y})$

The final position is:

$$P' = T(t_{2x}, t_{2y}) \cdot T(t_{1x}, t_{1y}) \cdot P$$

Multiplying the two translation matrices gives:

$$T(t_{2x}, t_{2y}) \cdot T(t_{1x}, t_{1y}) = T(t_{1x} + t_{2x}, t_{1y} + t_{2y})$$

Key Result

Translations are additive.

This matches physical intuition:

- Moving right by 3 units
- Then right by 5 units
- Is equivalent to moving right by 8 units

4. Composite Two-Dimensional Rotations

Rotation Recap

Rotation about the origin by angle θ :

$$R(\theta)$$

Two Successive Rotations

Apply:

1. Rotation $R(\theta_1)$
2. Rotation $R(\theta_2)$

Final transformation:

$$P' = R(\theta_2) \cdot R(\theta_1) \cdot P$$

Multiplying the matrices:

$$R(\theta_2) \cdot R(\theta_1) = R(\theta_1 + \theta_2)$$

Key Result

Rotations are additive in angle.

Rotating:

- 30° then 45°
- Is equivalent to rotating 75° once

5. Composite Two-Dimensional Scaling

Scaling Recap

Scaling by (s_x, s_y) :

$$S(s_x, s_y)$$

Two Successive Scalings

Apply:

1. Scaling $S(s_{1x}, s_{1y})$

2. Scaling $S(s_{2x}, s_{2y})$

Composite matrix:

$$S(s_{2x}, s_{2y}) \cdot S(s_{1x}, s_{1y}) = S(s_{1x}s_{2x}, s_{1y}s_{2y})$$

Key Result

Scaling factors multiply.

Example:

- Scale by 3
- Then scale by 3 again
- Final scale = 9

6. General Pivot-Point Rotation (Very Important)

Most graphics libraries provide:

- Rotation **only about the origin**

But in practice, we often want to rotate about a point:

$$(x_r, y_r)$$

Solution: Translate-Rotate-Translate

Steps:

1. Translate pivot point to origin

$$T(-x_r, -y_r)$$

2. Rotate about origin

$$R(\theta)$$

3. Translate pivot back

$$T(x_r, y_r)$$

Composite Matrix

$$R(x_r, y_r, \theta) = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$$

This produces a **single matrix** that:

- Rotates the object
- About the specified pivot point
- Without manual point-by-point computation

Many modern libraries do this internally.

7. General Fixed-Point Scaling

Scaling normally occurs about the origin, but often we want to scale about a **fixed point**:

$$(x_f, y_f)$$

Transformation Sequence

1. Translate fixed point to origin

$$T(-x_f, -y_f)$$

2. Scale

$$S(s_x, s_y)$$

3. Translate back

$$T(x_f, y_f)$$

Composite Matrix

$$S(x_f, y_f, s_x, s_y) = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f)$$

This ensures:

- Object scales relative to the fixed point
- Fixed point remains unchanged

8. Scaling Along Arbitrary Directions

So far:

- Scaling occurs along x and y axes only

But sometimes we want scaling along **arbitrary directions**.

Strategy

1. Rotate object so desired scaling directions align with x and y axes
2. Scale using $S(s_1, s_2)$
3. Rotate back to original orientation

Composite Matrix

$$R^{-1}(\theta) \cdot S(s_1, s_2) \cdot R(\theta)$$

This matrix:

- Scales by s_1 and s_2
- Along directions rotated by angle θ
- Without changing object orientation

Practical Example

- Stretch a square along its diagonal
 - Rotate diagonal to y-axis
 - Scale
 - Rotate back
 - Result: a parallelogram
-

9. Matrix Concatenation Properties

Associativity

Matrix multiplication is associative:

$$M_3 \cdot M_2 \cdot M_1 = (M_3 \cdot M_2) \cdot M_1 = M_3 \cdot (M_2 \cdot M_1)$$

This allows:

- Flexible grouping
 - Efficient computation of composite matrices
-

10. Order of Transformations Matters (Critical Concept)

Matrix multiplication is **not commutative**:

$$M_1 \cdot M_2 \neq M_2 \cdot M_1$$

Meaning in Graphics

Changing the order of transformations **changes the result**.

Example:

- Translate then rotate
- Rotate then translate

These produce **different final positions**.

Visual Interpretation

- Translation moves object along world axes
 - Rotation changes object orientation and coordinate directions
 - Reversing order alters motion path
-

11. When Transformations Are Commutative

Some special cases **are commutative**:

- Translation + translation
- Rotation + rotation
- Scaling + scaling
- Rotation + uniform scaling ($s_x = s_y$)

Outside these cases:

- **Order must be handled carefully**

12. Premultiplication vs Postmultiplication

Different graphics systems use different conventions:

Premultiplication

- New matrix multiplied on the **left**
- Transformations applied in the order specified

Postmultiplication

- New matrix multiplied on the **right**
- Transformations applied in reverse order

This is why:

- OpenGL and similar systems require careful ordering
- Understanding matrix order is essential for correct results

Final Conceptual Summary

Transformation Type	Composite Behavior
Translation	Additive
Rotation	Additive (angles)
Scaling	Multiplicative
Mixed transforms	Order-dependent
Matrix multiplication	Associative, not commutative

If you want next, I can:

- Work **numerical examples step-by-step**

- Explain **OpenGL transformation order**
- Show **common exam traps**
- Convert this into **short exam notes**
- Connect this to **3D transformations**

Just tell me how you want to continue.

Below is a **complete, exam-ready, concept-heavy explanation** of the section “**General Two-Dimensional Composite Transformations and Computational Efficiency**”, followed by a **cleanly rewritten and properly formatted version of the program**, with **clear intent and logic preserved**. I will explain **what, why, and how**, exactly in the spirit of Hearn & Baker.

1. General Two-Dimensional Composite Transformations

Any **combination of translation, rotation, and scaling in 2D** can be represented using **one single 3×3 homogeneous transformation matrix**.

The most general form is:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} rs_{xx} & rs_{xy} & tr_{sx} \\ rs_{yx} & rs_{yy} & tr_{sy} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Meaning of Matrix Elements

1. Rotation–Scaling Terms (multiplicative)

The four elements:

- $rs_{xx}, rs_{xy}, rs_{yx}, rs_{yy}$

represent the **combined effect of rotation and scaling**.

They depend only on:

- Rotation angle(s)
- Scaling factors

No translation information is stored here.

2. Translation Terms (additive)

The elements:

- tr_{sx}, tr_{sy}

contain **all translation-related effects**, including:

- Explicit translations
- Pivot-point offsets
- Fixed-point offsets
- Interaction terms from rotation + scaling

This is why translation cannot be treated independently once transformations are combined.

Example: Scale + Rotate about Centroid + Translate

Suppose an object is:

1. Scaled about its centroid (x_c, y_c)
2. Rotated about the same centroid by angle θ
3. Translated by (t_x, t_y)

The composite matrix is:

$$T(t_x, t_y) \cdot R(x_c, y_c, \theta) \cdot S(x_c, y_c, s_x, s_y) = \begin{bmatrix} s_x \cos \theta & -s_y \sin \theta & x_c(1 - s_x \cos \theta) + y_c s_y \sin \theta + t_x \\ s_x \sin \theta & s_y \cos \theta & y_c(1 - s_y \cos \theta) - x_c s_x \sin \theta + t_y \\ 0 & 0 & 1 \end{bmatrix}$$

This single matrix fully encodes the entire transformation sequence.

2. Computational Efficiency of Composite Transformations

Naive Approach (inefficient)

If transformations are applied **one by one**:

- Each step requires multiplications and additions
- Rotation requires trig calls
- Total operations per point increase significantly

Optimized Approach (used in graphics systems)

After concatenation, transformed coordinates are computed as:

$$x' = x \cdot r_{sx} + y \cdot r_{sy} + tr_{sx}$$

$$y' = x \cdot r_{sy} + y \cdot r_{sy} + tr_{sy}$$

Operation Count (per point)

- **4 multiplications**

- **4 additions**

This is the **maximum cost**, regardless of how many transformations were combined.

This is why:

- GPUs always use composite matrices
- OpenGL, DirectX, Vulkan work this way
- Scene graphs depend on this principle

3. Rotation Approximation for Small Angles

Rotation is computationally expensive due to:

- $\sin(\theta)$
- $\cos(\theta)$

Small-Angle Approximation

For angles $< 10^\circ$:

$$\cos \theta \approx 1 \quad \sin \theta \approx \theta \quad (\text{in radians})$$

Then rotation becomes:

$$x' = x - y \sin \theta$$

$$y' = x \sin \theta + y$$

Only:

- 2 multiplications
- 2 additions per point

Trade-off

- Error accumulates over many steps
- Must periodically reset object position
- Common resets: every 180° or 360°

This is heavily used in:

- Real-time animation
- Physics simulations
- Incremental rotations

4. Inverse Matrices in Composite Transformations

Many composite transformations require **inverse operations**, especially:

- Arbitrary-direction scaling
- Reflections
- Shears

Efficient Inverses

Transformation	Inverse
Translation	Negate translation values
Rotation	Transpose rotation matrix
Scaling	Reciprocal scale factors

These are **far cheaper** than computing general matrix inverses.

5. Two-Dimensional Rigid-Body Transformations

A **rigid-body transformation** includes:

- Rotation
- Translation
- No scaling

General form:

$$\begin{bmatrix} r_{xx} & r_{xy} & t_x \\ r_{yx} & r_{yy} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Properties

- Distances preserved
- Angles preserved
- Shape preserved

The upper-left 2×2 matrix is **orthogonal**.

Orthonormal Property

The rotation submatrix satisfies:

$$\begin{aligned} r_{xx}^2 + r_{xy}^2 &= 1 \\ r_{yx}^2 + r_{yy}^2 &= 1 \end{aligned}$$

$$r_{xx}r_{yx} + r_{xy}r_{yy} = 0$$

Meaning:

- Rows (or columns) are unit vectors
- They are perpendicular

This is why rotations do not distort objects.

6. Constructing Rotation Matrices from Orientation Vectors

Sometimes we know:

- Final orientation
- But not the rotation angle

If unit vectors **u** and **v** describe the desired orientation:

- Put **u** as first row
- Put **v** as second row

This constructs the rotation matrix directly.

Used in:

- Camera alignment
- Object-to-object alignment
- Coordinate system conversion

7. Composite-Matrix Programming Example (Explained)

The program demonstrates:

Transformation Sequence

1. Scale triangle about centroid
2. Rotate about centroid
3. Translate upward

Strategy

- Start with identity matrix
- Each transformation **premultiplies** the composite matrix
- Final matrix applied once to all vertices

8. Clean, Properly Formatted Version of the Code

Below is the **same program**, cleaned, readable, and logically grouped.

```
cpp

#include <GL/glut.h>
#include <cmath>

/* ----- GLOBALS ----- */
typedef GLfloat Matrix3x3[3][3];

struct wcPt2D {
    GLfloat x, y;
};

Matrix3x3 compMatrix;
const double PI = 3.141592653589793;

/* ----- MATRIX UTILITIES ----- */
void setIdentity(Matrix3x3 m) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            m[i][j] = (i == j);
}

void preMultiply(Matrix3x3 m1, Matrix3x3 m2) {
    Matrix3x3 temp;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            temp[i][j] = m1[i][0]*m2[0][j] +
                        m1[i][1]*m2[1][j] +
                        m1[i][2]*m2[2][j];

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            m2[i][j] = temp[i][j];
}

/* ----- TRANSFORMATIONS ----- */
void translate2D(float tx, float ty) {
    Matrix3x3 t;
    setIdentity(t);
    t[0][2] = tx;
    t[1][2] = ty;
    preMultiply(t, compMatrix);
}

void rotate2D(wcPt2D pivot, float theta) {
    Matrix3x3 r;
    setIdentity(r);

    r[0][0] = cos(theta);
    r[0][1] = -sin(theta);
    r[0][2] = pivot.x * (1 - cos(theta)) + pivot.y * sin(theta);

    r[1][0] = sin(theta);
    r[1][1] = cos(theta);
    r[1][2] = pivot.y * (1 - cos(theta)) - pivot.x * sin(theta);

    preMultiply(r, compMatrix);
}

void scale2D(float sx, float sy, wcPt2D fixed) {
    Matrix3x3 s;
    setIdentity(s);

    s[0][0] = sx;
    s[1][1] = sy;
    s[0][2] = fixed.x * (1 - sx);
    s[1][2] = fixed.y * (1 - sy);
}
```

```

    preMultiply(s, compMatrix);
}

/* ----- APPLY TRANSFORMATION ----- */
void transformVerts(int n, wcPt2D *v) {
    for (int i = 0; i < n; i++) {
        float x = v[i].x;
        float y = v[i].y;

        v[i].x = compMatrix[0][0]*x + compMatrix[0][1]*y + compMatrix[0][2];
        v[i].y = compMatrix[1][0]*x + compMatrix[1][1]*y + compMatrix[1][2];
    }
}

```

9. Key Takeaways (Very Important)

- Any number of transformations → **one matrix**
- Composite matrices drastically reduce computation
- Rigid-body transformations preserve shape
- Order of multiplication **matters**
- Homogeneous coordinates make this possible
- This is the foundation of **all modern graphics pipelines**

If you want next, I can:

- Convert this into **exam short notes**
- Explain **3D extension of rigid-body transforms**
- Show **OpenGL transformation stack mapping**
- Give **numerical solved problems**

Just tell me.

Below is a **brief, clear explanation** of the **other two-dimensional transformations: Reflection and Shear**, focusing on **what they do, how they work, and where they are used**, without going into derivations or long proofs.

1. Reflection (2D)

What is Reflection?

A **reflection** produces a **mirror image** of an object with respect to a specified line (axis of reflection).

Conceptually:

- Reflection is equivalent to a **180° rotation about the reflection axis**

- Shape and size are preserved
- Orientation is reversed (clockwise ↔ counterclockwise)

Common Types of Reflections

1. Reflection about the x-axis ($y = 0$)

- x remains the same
- y changes sign

Matrix:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Effect: Object is flipped vertically.

2. Reflection about the y-axis ($x = 0$)

- y remains the same
- x changes sign

Matrix:

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Effect: Object is flipped horizontally.

3. Reflection about the origin

- Both x and y change sign

Matrix:

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Equivalent to:

- Reflection about x-axis followed by y-axis
- Rotation by 180° about the origin

4. Reflection about the line $y = x$

- x and y coordinates are swapped

Matrix:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Achieved using:

- Rotate -45°
- Reflect about x-axis
- Rotate $+45^\circ$

5. Reflection about arbitrary lines ($y = mx + b$)

Performed using a **sequence**:

1. Translate line to pass through origin
2. Rotate line onto x- or y-axis
3. Reflect
4. Apply inverse rotation
5. Apply inverse translation

Important Notes on Reflection

- Reflection can be implemented using **negative scaling**
- Scaling factors $\neq \pm 1$ can also:
 - Enlarge
 - Reduce
 - Distort reflected objects

2. Shear (2D)

What is Shear?

A **shear** distorts an object so that:

- One coordinate shifts in proportion to the other
- Shape changes, but **area may be preserved**
- Parallel lines remain parallel

Visually:

- Squares become parallelograms
- Looks like layers sliding over each other

1. X-Direction Shear

Shear relative to x-axis

Matrix:

$$\begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Equations:

$$x' = x + sh_x \cdot y$$

$$y' = y$$

Effect:

- Points move horizontally
- Amount depends on y-value

X-Shear relative to line $y = y_{ref}$

Matrix:

$$\begin{bmatrix} 1 & sh_x & -sh_x y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Equation:

$$x' = x + sh_x(y - y_{ref})$$

Used when shear must occur about a specific reference line.

2. Y-Direction Shear

Shear relative to y-axis

Matrix:

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Equations:

$$x' = x$$

$$y' = y + sh_y \cdot x$$

Y-Shear relative to line $x = x_{ref}$

Matrix:

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$

Equation:

$$y' = y + sh_y(x - x_{ref})$$

Key Properties of Shear

- Shear does **not preserve angles**
- Lengths parallel to the reference axis remain unchanged
- Can be expressed as **composite transformations**
- Translation is equivalent to shear relative to a shifted reference line

3. Summary Table

Transformation	Preserves Shape	Preserves Size	Changes Orientation
Reflection	Yes	Yes	Yes (mirrored)
Shear	No	Area sometimes	Yes

4. Where These Are Used

- Reflection: symmetry, mirroring, coordinate conversion
- Shear: italic text, slanted shapes, projection effects
- Both are implemented using **homogeneous matrices**
- Often built using **composite transformations**

If you want, I can:

- Convert this into **exam-point notes**
- Give **one-line definitions + matrices**
- Provide **numerical examples**

- Compare **shear vs scaling**
- Connect to **3D reflections and shears**

Just tell me.

Below is a **clear, structured, and detailed explanation** of the full topic “**Transformations between Two-Dimensional Coordinate Systems**”, followed by **OpenGL’s role in 2D transformations**, exactly aligned with how this material is expected to be understood in **computer graphics theory, exams, and practical APIs**.

1. Why Coordinate-System Transformations Are Needed

In computer graphics, objects are **not always defined in one global coordinate system**.

Typical situations:

- **Modeling:** Each object (chair, table, machine part) is defined in its **own local coordinate system**
- **Scene assembly:** Objects must be placed, oriented, and scaled in a **world coordinate system**
- **Viewing pipeline:** World coordinates must be transformed into **device/screen coordinates**
- **Non-Cartesian descriptions:** Some objects are defined in polar, spherical, or other coordinate systems

Therefore, we frequently need to:

Transform coordinates from one 2D Cartesian reference frame to another

2. Description of Two 2D Cartesian Coordinate Systems

Consider:

- A **global reference frame:** xy
- A **local reference frame:** $x'y'$

The local system is defined by:

- An origin at (x_0, y_0) in the global system
- An orientation angle θ relative to the global axes

This situation is shown conceptually in Figure 30 of the text.

The goal is to **convert object coordinates defined in the local system into global coordinates**, or vice versa.

3. Transforming from One 2D Cartesian System to Another

To transform coordinates from the global xy system into the local $x'y'$ system, we must **superimpose the two coordinate frames**.

This is done using **two basic steps**.

Step 1: Translation of the Origin

We translate the coordinate system so that the origin (x_0, y_0) of the local frame moves to the origin $(0, 0)$ of the global frame.

Translation matrix:

$$T(-x_0, -y_0) = \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

After this step:

- The origins coincide
 - The axes are still misaligned
-

Step 2: Rotation to Align Axes

Next, we rotate the translated system so that:

- The local x' axis aligns with the global x axis

This is achieved by a **clockwise rotation** of angle θ , i.e., a rotation by $-\theta$.

Rotation matrix:

$$R(-\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composite Transformation Matrix

Combining both steps gives the **complete transformation**:

$$M_{xy \rightarrow x'y'} = R(-\theta) \cdot T(-x_0, -y_0)$$

This single matrix:

- Translates the origin
 - Rotates the axes
 - Converts coordinates between the two systems
-

4. Alternative Orientation Specification Using Direction Vectors

Instead of specifying orientation using an angle θ , we can describe the local coordinate system using a **direction vector**.

Using a Direction Vector V

Let:

- $V = (V_x, V_y)$ define the direction of the **positive y' axis**

Convert it to a unit vector:

$$\mathbf{v} = \frac{V}{|V|} = (v_x, v_y)$$

The corresponding **x' axis unit vector** is obtained by a 90° clockwise rotation:

$$\mathbf{u} = (v_y, -v_x)$$

Constructing the Rotation Matrix from Unit Vectors

From earlier theory, we know:

- The rows (or columns) of a rotation matrix form an **orthonormal basis**

Thus, the rotation matrix can be written directly as:

$$R = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This approach is especially useful when:

- Orientation is defined by alignment constraints
- Angle values are not explicitly known
- Objects must face a direction or another object

5. Orientation from Two Points (Interactive Applications)

In interactive systems, orientation is often defined using **two points**:

- P_0 : origin of the local system
- P_1 : a point that defines the direction of the y' axis

The direction vector is:

$$\mathbf{v} = \frac{P_1 - P_0}{|P_1 - P_0|}$$

Then:

- $\mathbf{u}$ is computed as a perpendicular to $\mathbf{v}$
- The rotation matrix is constructed exactly as above

This method is widely used in:

- CAD systems
- Robotics
- Scene graphs
- Camera alignment

6. OpenGL and Two-Dimensional Transformations

Although OpenGL is a **3D graphics API**, it supports 2D transformations naturally by:

- Using 4×4 matrices
- Setting the z component appropriately

Internally:

- Coordinates are 4-element homogeneous vectors
- Transformations are stored as **composite matrices**

7. Basic OpenGL Transformation Functions

Translation

```
c
glTranslatef(tx, ty, tz);
```

- For 2D: set $tz = 0$
- Translates all subsequent geometry

Example:

```
c
glTranslatef(25.0, -10.0, 0.0);
```

Rotation

```
c
glRotatef(theta, vx, vy, vz);
```

- **theta** is in degrees
- (**vx**, **vy**, **vz**) defines the rotation axis
- For 2D rotation: rotate about the z-axis

Example:

```
c
glRotatef(90.0, 0.0, 0.0, 1.0);
```

Internally:

- OpenGL uses **quaternions** for efficient arbitrary-axis rotations

Scaling

```
c
glScalef(sx, sy, sz);
```

- For 2D: set **sz** = 1
- Negative scale values produce reflections

Example:

```
c
glScalef(2.0, -3.0, 1.0);
```

8. OpenGL Matrix Modes

OpenGL maintains **separate matrix stacks**.

To apply geometric transformations:

```
c
glMatrixMode(GL_MODELVIEW);
```

Key matrix functions:

Reset to Identity

```
c
glLoadIdentity();
```

Load Custom Matrix

```
c
glLoadMatrixf(elements16);
```

- Matrix must be specified in **column-major order**

Concatenate Matrix

```
c
glMultMatrixf(elements16);
```

- Postmultiplies the current matrix

9. Order of Transformations in OpenGL (Critical Concept)

In OpenGL:

- Transformations are **postmultiplied**
- The **last transformation specified is applied first**

Example:

```
c
glLoadIdentity();
glTranslatef(...);
glRotatef(...);
```

Effectively applies:

1. Rotation
2. Then translation

This is why OpenGL transformations often feel “reversed” compared to mathematical notation.

10. Example: Applying Individual Transformations

In the provided example:

1. A blue rectangle is drawn with the identity matrix
2. A translation is applied → red translated rectangle
3. Matrix reset
4. Rotation applied → red rotated rectangle
5. Matrix reset
6. Scaling + reflection applied → transformed rectangle

Each transformation is **isolated** by resetting the matrix.

11. Key Conceptual Summary

- Objects are often defined in **local coordinate systems**
 - Coordinate-system transformations use:
 - Translation to align origins
 - Rotation to align axes
 - Homogeneous coordinates allow all transformations to be expressed as matrices
 - Composite matrices improve efficiency
 - OpenGL uses:
 - 4×4 matrices
 - Column-major storage
 - Reverse application order
 - Understanding matrix order is essential to avoid incorrect transformations
-

Final Takeaway

Transformations between coordinate systems are the **foundation of modeling, viewing, and animation pipelines**.

Mastery of translation–rotation composition and matrix order is essential for both **theoretical understanding** and **correct OpenGL programming**.

If you want, I can:

- Reduce this into **exam-point notes**
- Provide **step-by-step numerical examples**
- Map this directly to **3D coordinate-system transformations**
- Explain **camera/view transformations next**

Just tell me how you want to continue.